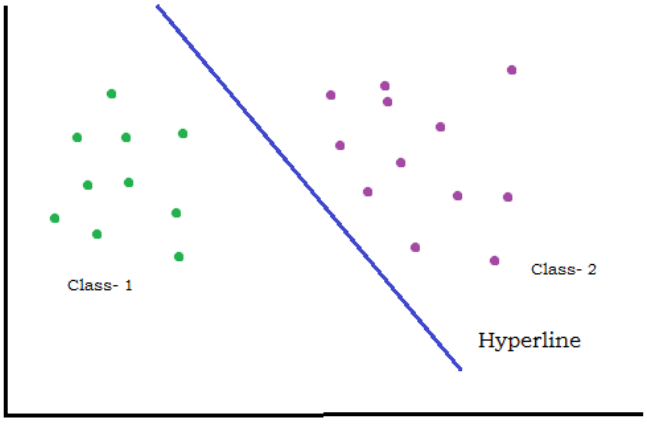
Support Vector Machine

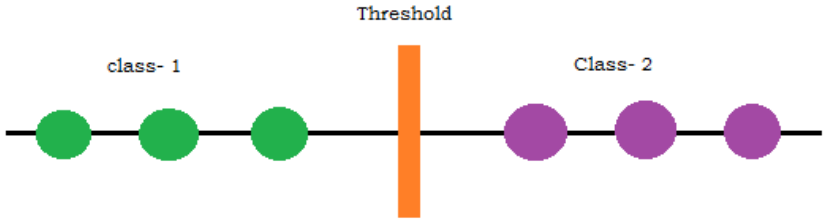
In machine learning algorithms SVMs (Support Vector Machines) are the most advanced algorithms and more specifically for classification to give the best prediction for the classes to be nicely distributed. Eventually it classifies the classes of data into n different parts. If I elaborate the point, I would say, it divides the data into two different parts for actual analysis of the prediction. As like we do a process like ROC and AUC to get the best threshold values for the best prediction, here the same we do classify the data in more appropriate way so that we do not face a huge error. A question definitely arises; why are we using these algorithms thou we have logistic regression and other algorithms as well? The simplest answer I would give that; each model has its own advantage and disadvantage. And definitely we prefer this model for classification over other techniques, and then definitely it has some more advantages over other algorithms.

As the same work is done by threshold separation for estimation of the classes, this technique also does the same things, but here it has many advantages over others. Let's discuss this step by step.

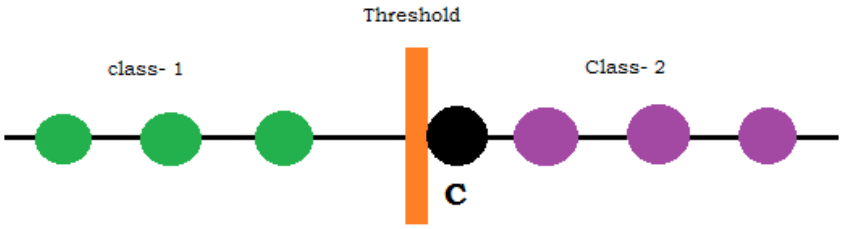
If suppose the data is 2D, and we have the following plot for the classification



In above diagram we have two classes of the same dataset. We have predicted the distinct classes. But those classes were predicted by the separation line. That line also called as **Hyperline** or **Decision Boundary**. We can also see this diagram with the data distribution like below:



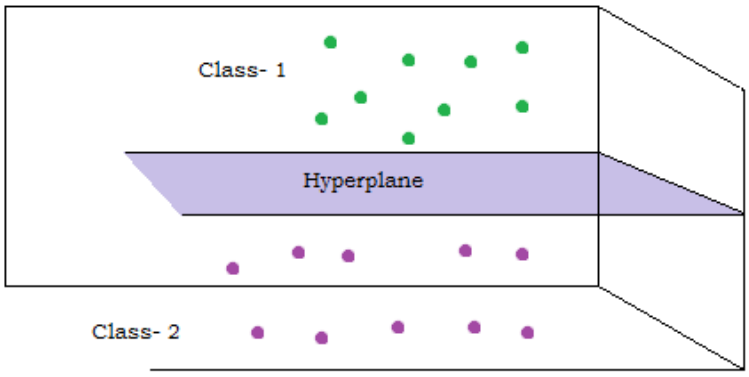
As we have the data predicted the classes for the two different classes. The same thing is done above in 2D plot, for classification of prediction. Let suppose the data is like this:



One question I would ask to you that, **in which class you is going to set the point C into the separation?** Of course this is a confusing thing where we are facing specificity which leads to the error. Hence for manage such thing we are about to deal with SVM technique. Even this has some advance algorithms like **Kernel Functions**.

More precisely we say; it has many different kinds to deal with multidimensional data and multiple classes as well. Because of these issues it is quite sensible that, we cannot deal with any simpler model or algorithms.

Machine separates the classes into 2 or many sub classes to predict the class to reduce the overfitting in the model.

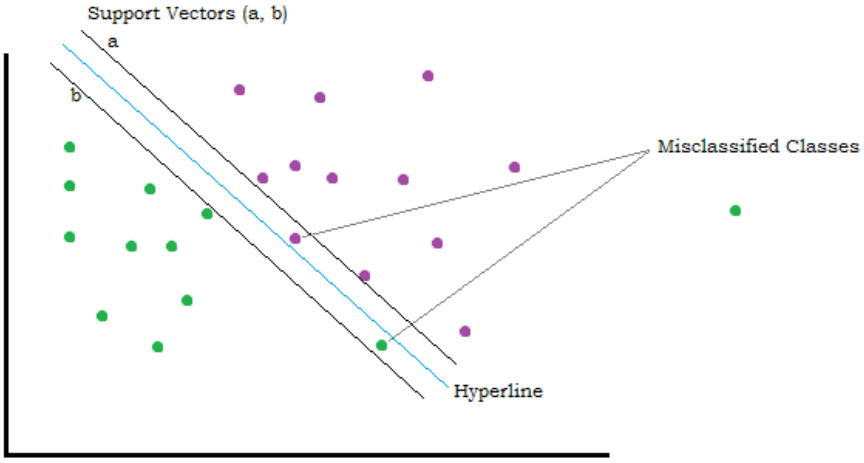


For 2D we distribute the classes by hyperline, as I discussed above. But what if the data is 3D? hence 3D plot is distributed by hyperplane. This seems more interesting for elaboration of the classes. But the question is resisting our imagination that, where we supposed to take the hyperline or hyperplane? This actual technical is known as SVM. Thus this machine handles such things. Basically it handles two things:

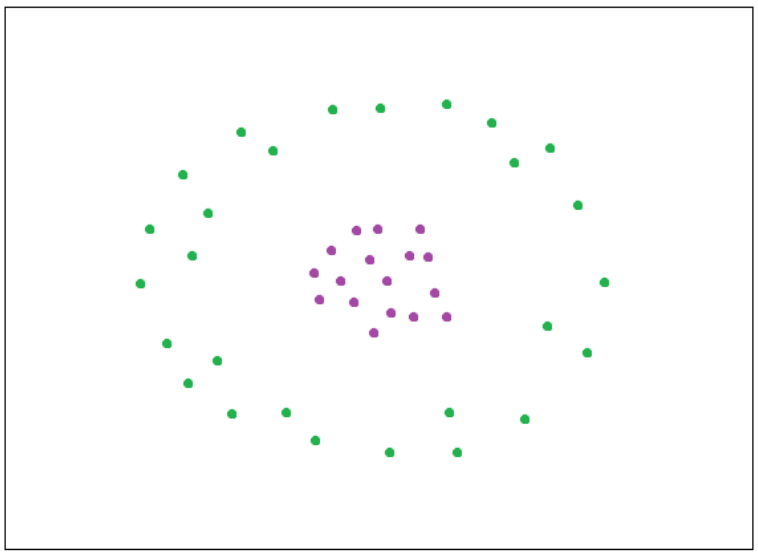
- Outliers in the data
- Overlapping of the classes.

What does mean overlapping of the classes? Overlapping means, actually the class seems to be existing into class- 1 but it exists into class- 2. This is overlapping. SVM draws the most optimal line so that the classes get distributed so sincerely so that we get a good estimation.

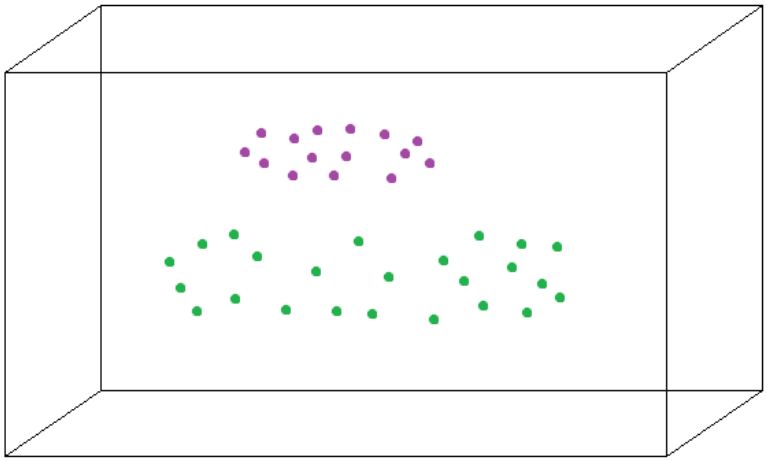
We plot the two vectors which are drawn with respect to the closest points of two different classes. Hence we approach by these lines. These lines are called Support Vectors. These lines are parallel. Again if we face misclassified classes as shown into the plot, then we eventually work with cross validation, you may be familiar with.



Yet we have dealt with linear data, where the change seems to be linear, hence we distributed the data into different classes. What if we have non-linear data? Are we going to use the same technique as that of linear? Of course it is not. Then can we deal with such scenario? Yes!!



It seems to be logical that, we neither draw simple line nor plane. Then the technique we use here is **Kernel functions**, what I just mentioned above. This technique handles this scenario automatically. We don't need to worry to deal with optimal line or plane. Let's see how it deals with such scenario?

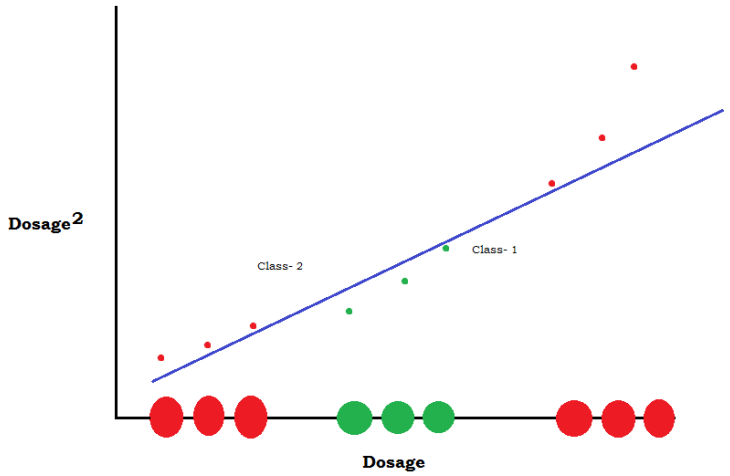


This is how it deals with the non-linear data. It separates in the same complicated way to the data. Let's see an example of such non-linear data and its handling by SVM.

Let suppose we have the data of the people cured by the medicines and we have the quantity of the dosage, where we predict whether the person cured. We have the following scenario:

We know neither high dose cures not low does. We must have to take accurate and intermediate. Hence here as well we cannot do hyperline so easily. So we go with Kernel functions. What will be happening here then? Any one of the functions will do transformation with 1 more dimension. Here we have 1D; function will convert this into 2D.

One of the techniques will do some transformation into the sets of classes, what it is shown into the following plot. Hence we can deal with scenarios along an appropriate technique.



This how; we do deal with the scenario. Not necessary that all the times Kernel Functions does transformations. It has multiple processes, just because we cannot in which dimension the data does exist.

Now I am curious to answer the answer of the question what I asked above, why do we need these algorithms? Thou the answer is; SVMs do not allow misclassifications for the data to predict appropriate the class.

Main idea behind Support Vector Machine:

- Start the data in relatively one dimension. Hence if necessary it will be automatically about to deal with 2 or more dimensions.
- Find the support vector classifier that separates the data into two or more groups. If we have higher dimensions, it also separates the data into groups either by transformation or any other technique.

How transformation is done by Kernel Functions?

Kernel Functions are used to manipulate the data; it generally transforms the training data so that non-linear decision is able to transform to a linear line of multi-dimensional space data.

Python Codes:

Scikit-learn have all the packages of the Kernel Functions. Essentially install this library into the notebook, and then proceed.

```
pip install scikit-learn
```

- Polynomial Kernel Function

```
from sklearn.svm import SVC
classifier = SVC(kernel ='poly', degree = 4)
classifier.fit(x_train, y_train)
```

- Gaussian Kernel/ Radial Bias Function (RBF) Kernel

```
from sklearn.svm import SVC
classifier = SVC(kernel ='rbf', random_state = 0)
classifier.fit(x_train, y_train)
```

- Sigmoid Kernel

```
from sklearn.svm import SVC
classifier = SVC(kernel ='sigmoid')
classifier.fit(x_train, y_train)
```

- Linear Kernel

```
from sklearn.svm import SVC
classifier = SVC(kernel ='linear')
classifier.fit(x_train, y_train)
```

ASAD ASHRAF KAREL